33

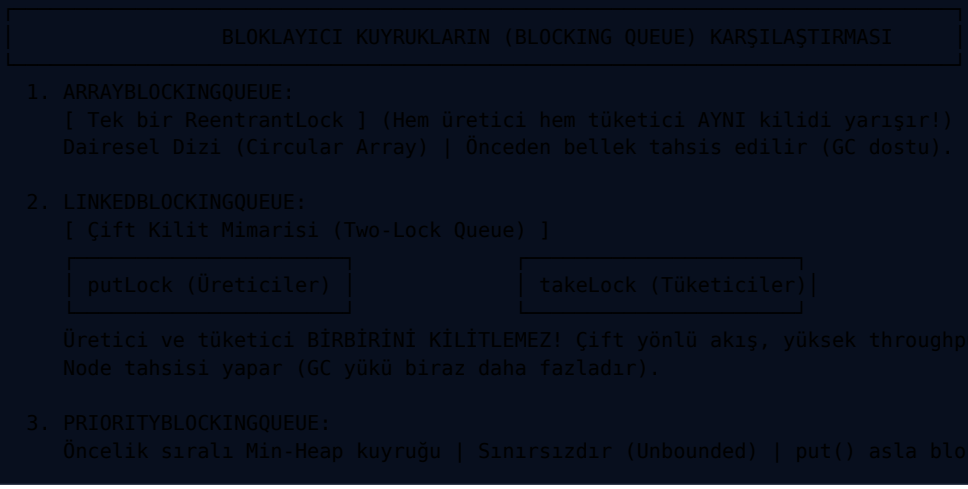
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Bloklayıcı Kuyruklar: ArrayBlockingQueue vs LinkedBlockingQueue vs PriorityBlockingQueue

Blocking Queues: ArrayBlockingQueue vs LinkedBlockingQueue vs PriorityBlockingQueue

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Fabrika çıkışındaki 3 farklı konveyör bandı gibidir: 1) ArrayBlockingQueue: Sabit 10 kutuluk metal bir tepsidir. Tepsi baştan ayrılmıştır, yeri sabittir. Ama tepsinin başında tek bir kilit vardır; aşçı yemek koyarken garson yemek alamaz. 2) LinkedBlockingQueue: Çift kapılı tüneldir. Tünelin girişinde ayrı kilit (putLock), çıkışında ayrı kilit (takeLock) vardır! Aşçı arkadan yemekleri yüklerken garson önden tepsileri alır; birbirlerinin ayağına basmazlar! 3) PriorityBlockingQueue: Acil servis kuyruğudur; durumu en acil olan hasta sıranın en önüne geçer!

🇬🇧 ENGLISH (ELI5)

Compare three factory conveyor belts: 1) ArrayBlockingQueue: A fixed 10-slot circular steel tray. Pre-allocated and bounded. However, it relies on a SINGLE lock shared between producers and consumers. 2) LinkedBlockingQueue: A two-door tunnel using 'Two-Lock Queue' topology. Producers contend for `putLock` at the entrance; consumers contend for `takeLock` at the exit. Producers and consumers operate concurrently without blocking each other! 3) PriorityBlockingQueue: A hospital triage queue where highest-priority elements jump to the head of the line via a binary heap.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

İki Kilitli Kuyruk Algoritması (Two-Lock Queue): 1) LinkedBlockingQueue'nun Başarımı: H.S. Chao ve Doug Lea'nın uyguladığı bu algoritmada, baş (head) ve kuyruk (tail) düğümleri iki ayrı `ReentrantLock` (`putLock` ve `takeLock`) ile korunur. Üreticiler sadece kuyruğun sonuna eleman eklerken, tüketiciler kuyruğun başından çeker. Aralarındaki tek bağ `AtomicInteger count` değişkenidir. Bu sayede eşzamanlı okuma ve yazma işlemlerinde `ArrayBlockingQueue`'dan 2-3 kat daha yüksek throughput üretir. 2) Bellek Farkı: ArrayBlockingQueue diziyi baştan tek parça tahsis eder, sıfır GC çöpü üretir. LinkedBlockingQueue her eleman için bir `Node` nesnesi tahsis eder; yüksek frekansta Young Generation GC baskısı yaratır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Two-Lock Queue Mechanics & Heap Trade-Offs: 1) LinkedBlockingQueue Concurrency Isolation: Employs a variation of the Michael & Scott Two-Lock Queue algorithm. Enqueue operations synchronize exclusively on `putLock` modifying `last`; dequeue operations synchronize exclusively on `takeLock` modifying `head`. Atomicity of capacity is preserved via `AtomicInteger count`. Producers and consumers rarely serialize against one another. 2) Garbage Collection Overhead: `ArrayBlockingQueue` allocates an `Object[]` array upfront, maintaining zero allocation churn at runtime. `LinkedBlockingQueue` instantiates a `Node\<E>` wrapper for every single item enqueued, generating GC churn in high-throughput allocations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: LinkedBlockingQueue ile Yüksek Hızlı Producer-Consumer Pipeline
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;

public class HighSpeedPipeline {
    void main() throws InterruptedException {
        // ⚠ Üretimde mutlaka bounded kapasite verilmelidir!
        var queue = new LinkedBlockingQueue<String>(10_000);

        // Üretici (Producer) - putLock üzerinde çalışır
        Thread.ofPlatform().name("IngestionWorker").start(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    queue.put("Mesaj-" + i);
                    IO.println("Kuyruğa eklendi: Mesaj-" + i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Tüketici (Consumer) - takeLock üzerinde çalışır (Çakışma YOK!)
        Thread.ofPlatform().name("ExportWorker").start(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    String msg = queue.poll(1, TimeUnit.SECONDS);
                    IO.println("Kuyruktan işlendi: " + msg);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'ArrayBlockingQueue mu yoksa LinkedBlockingQueue mu daha az bellek tüketir ve GC dostudur?' diye sorar. Yanıt: 'ArrayBlockingQueue çok daha GC dostudur çünkü baştan tek bir dizi tahsis eder, çalışma anında hiç yeni nesne (Node) üretmez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Interview Nuance: 'Which queue exhibits superior GC profile under extreme load?' Senior Answer: 'ArrayBlockingQueue! It allocates its backing array once during construction, generating zero runtime node allocations. LinkedBlockingQueue allocates a Node instance for every put, inducing significant allocation churn.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ArrayBlockingQueue tek kilit kullanır; LinkedBlockingQueue çift kilit (put/take) kullanır.
- LinkedBlockingQueue üretici ve tüketiciyi ayırdığı için daha yüksek throughput sunar.
- ArrayBlockingQueue baştan dizi tahsis ettiği için sıfır GC çöpü üretir.

🎯 Key Takeaways for the Interview

- ArrayBlockingQueue uses a single lock; LinkedBlockingQueue uses independent put/take locks.
- Dual locks grant LinkedBlockingQueue higher concurrency by decoupling producers from consumers.
- ArrayBlockingQueue avoids GC churn by reusing its preallocated array storage.